

Recognizing the 18 Patterns

Interview study guide. The goal isn't to solve — it's to look at a problem and instantly say "that's a ____." Read this like flashcards.

The whole skill in one shift: stop asking "how would I solve this?" (that's inventing brute force). Ask "which of my 18 shapes is this?" (that's recognizing). The reflex:

① **bound on n** → allowed complexity (kills brute force) ② **cue words** → 2–3 candidates ③ **a constraint** kills the impossible ones (e.g. negatives kill sliding window).

THE LEAK THIS REVISION FIXES — DRILL, AUG 19

Scored  1 /  3 /  2 over six reps. In **three** of them the *mechanism described was correct* and the *name attached to it was wrong* — a heap called "sliding window," a binary-search-on-answer called "monotonic stack." The reasoning is intact; the **vocabulary layer** is what fails under a clock.

Two documentation bugs caused the rest: 13 of the old 15 patterns stated *no complexity at all* (so complexity got guessed — two-pointer called $O(\log n)$), and **space** was never volunteered once in six reps. Every pattern below now carries time *and* space.

Drill it as: say the pattern *name*, its *number*, then time *and* space — out loud, in that order, every rep. The name is the part that isn't firing.

Constraint → allowed complexity (read this BEFORE the problem)

The bound on n is the answer key to the complexity. Work backwards from it. Budget $\sim 1e8$ simple ops/sec.

BOUND ON N	ALLOWED TIME	LIKELY APPROACH
$n \leq 10-12$	$O(n!)$ / $O(2^n \cdot n)$	brute force, permutations, backtracking
$n \leq 20-25$	$O(2^n)$	subset enumeration, bitmask DP, meet-in-middle
$n \leq 100-500$	$O(n^3)$	Floyd–Warshall, 3 nested loops, interval DP
$n \leq 2,000-5,000$	$O(n^2)$	2 nested loops, basic DP table, edit distance
$n \leq 1e5-1e6$	$O(n \log n)$ / $O(n)$	sort, two-pointer, sliding window, hashing, heap
$n \leq 1e7-1e8$	$O(n)$	single pass, prefix sums, counting
$n \leq 1e9-1e18$	$O(\log n)$ / $O(1)$	binary search, math/closed form, fast exponent

Watch which quantity the log runs over. If $n \leq 1e4$ but a *value* range is $1e9$, the log is over the **answer range R**, not n — that's $O(n \log R)$, not $O(\log n)$. Saying it that way proves you know which is which.

The 18 patterns — cue · what it does · the tell · cost

1. Two-pointer. CUE: sorted (or sortable) array, "find a pair/triple", "is there an $x+y = t$ ".

DOES: one index from each end, walk them toward each other.

TELL: sorted + looking for a relationship between two elements. *Ex: Two Sum on a sorted array → lo/hi pointers, sum too small move lo up.*

WHY $O(N)$: each step moves one pointer inward and neither ever backs up, so total movement is bounded by n . **It is a linear walk, not a halving** — this is *not* $O(\log n)$.

$O(n)$ time / $O(1)$ space ← the $O(1)$ is the selling point over a hash set

2. Sliding window. CUE: "longest/shortest **contiguous** subarray/substring with a property", values **non-negative**.

DOES: grow a window from the right, shrink from the left, keep a running quantity.

TELL: contiguous + the running quantity changes *monotonically with window size* (adding never decreases it, removing never increases it). *Ex: longest substring with all-distinct chars; shortest subarray with sum $\geq S$ (positive ints).*

WHY $O(N)$: each index enters the window once and leaves once.

$O(n)$ time / $O(k)$ space — $O(1)$ for a bounded alphabet

3. Prefix-sum array (range queries). CUE: many range-sum queries on a **fixed** array, "sum of range $[i, j]$ ", 2D rectangle sums, "immutable". The endpoints are **known**.

DOES: precompute running totals once; any range sum = $S[j] - S[i]$ in $O(1)$.

TELL: repeated range sums, 2D grid rectangle sums, or many range-add updates (difference array — the inverse trick). *You already know the endpoints; you're not searching for them.*

$O(n)$ build, $O(1)$ per query / $O(n)$ space

4. Prefix sum + hashmap. CUE: "contiguous subarray summing to **exactly k**", especially when values can be **negative**.

DOES: running prefix sum S ; count earlier prefixes equal to $S - k$ via a hashmap.

TELL: "sum = k" + endpoints *unknown* (and negatives *break* the window, so you fall back to this). *Seed the map with $\{0:1\}$ so a prefix that is k counts itself.*

THE MAP IS THE PATTERN. A prefix-sum *vector* alone still needs to compare every pair — that's $O(n^2)$. The hashmap is what buys the $O(n)$.

$O(n)$ time / $O(n)$ space

5. Hashmap / frequency array. CUE: "count occurrences", "seen before", "distinct", "anagram", "duplicate".

DOES: map value → count (or last-seen index). $O(1)$ lookup.

TELL: you need to remember what you've seen. *Bounded values (e.g. lowercase letters) → use a plain `int freq[26]`, not a hashmap: no hashing, no allocation, $O(1)$ space.*

NEVER SAY "SORTED HASHMAP." A hashmap has no order and needs none — you only ask it *count of this key* and *how many keys are live*.

$O(n)$ time / $O(n)$ space — $O(1)$ when the value range is bounded

6. Sorting + sweep. CUE: "intervals/meetings overlap or merge", "schedule", anything about order on a number line.

DOES: sort by start, then walk once keeping a "current" interval.

TELL: the word "interval" almost always = sort first. *Ex: merge overlapping intervals.*

$O(n \log n)$ time (the sort dominates) / $O(1)$ space beyond the sort

7. Binary search (on a sorted array). CUE: already sorted + "find / insert position of a value".

DOES: halve the search range each step.

TELL: sorted data + you're locating a specific value. Use $mid = lo + (hi-lo)/2$ — $(lo+hi)/2$ overflows.

$O(\log n)$ time / $O(1)$ space

8. Binary search on the answer. CUE: "min/max value such that a property holds", "is it possible within budget K?", and the property is **monotonic** (once true, stays true).

DOES: binary-search the *answer space*, test feasibility at each midpoint.

TELL: huge answer range + a cheap yes/no feasibility check. Ex: *minimum ship capacity to deliver in D days; slowest machine speed that still makes the deadline.*

INSIGHT TO SAY OUT LOUD: the yes/no test flips NO→YES exactly once across the range — bisect for the flip point.

$O(n \log R)$ time / $O(1)$ space — R = answer range, NOT n

9. Heap (priority queue). CUE: "k-th largest/smallest", "top k", "running median", "merge k lists".

DOES: keep only the k elements that matter.

TELL: you want the best few, not a full sort. *Sorting also works and is a fine first answer — say it, then offer the faster gear.*

FOR K LARGEST USE A MIN-HEAP (and vice versa): its top is the smallest of the k biggest — i.e. the running k-th largest, and exactly the element to evict.

heap $O(n \log k)$ / $O(k)$ · sort $O(n \log n)$ / $O(1)$ · quickselect $O(n)$ avg, $O(n^2)$ worst / $O(1)$

10. Monotonic stack. CUE: "next greater/smaller element", "span", "largest rectangle in histogram".

DOES: a stack kept in increasing/decreasing order; pop while the new element breaks the order.

TELL: "next/previous greater or smaller" is the dead giveaway. **It compares elements to each other** — if you're testing candidate *answers* instead, you want #8.

WHY $O(N)$: each index is pushed and popped at most once.

$O(n)$ time / $O(n)$ space

11. Stack. CUE: "balanced", "valid parentheses", nesting, undo, "evaluate expression".

DOES: push openers, pop/match on closers; must be empty at the end.

TELL: matching/nesting that must close in reverse order. C++: `std::stack<char> st;`

$O(n)$ time / $O(n)$ space

12. BFS (breadth-first search). CUE: "minimum steps/moves to reach", unweighted shortest path, "levels", grid distances.

DOES: explore level by level with a queue; first arrival = fewest steps.

TELL: graph/grid + *shortest*. Queue.

$O(V+E)$ time / $O(V)$ space

13. DFS / union-find. CUE: "connected components", "flood fill", "number of islands", "all paths", cycle detection.

DOES: go deep then backtrack (recursion/stack); union-find for grouping.

TELL: graph/grid + *reach/group everything* (not shortest).

DFS $O(V+E)$ / $O(V)$ · union-find $\sim O(\alpha(n))$ per op, effectively $O(1)$

14. Dynamic programming. CUE: "number of ways", "min/max cost/length", "can you make X", overlapping subproblems, small-ish n with combinatorial feel.

DOES: define a state + a transition; fill a table.

TELL: the answer depends on answers to smaller versions of the same problem. *Ex: coin change, edit distance, longest increasing subsequence.*

$O(\text{states} \times \text{transition cost})$ time / $O(\text{states})$ space – often trimmed to $O(\text{one row})$ with a rolling array

15. Math / bit trick. CUE: huge n with arithmetic, divisibility/parity, "without extra space", "single number".

DOES: closed-form formula, GCD, XOR tricks, sieve.

TELL: n is astronomically large ($1e9$ – $1e18$) so only $O(\log n)/O(1)$ survives → must be math.

$O(\log n)$ or $O(1)$ time / $O(1)$ space

16. Tree recursion. [NEW] CUE: the word "tree" at all – "depth", "height", "diameter", "lowest common ancestor", "is it balanced", "path sum".

DOES: recurse to children, combine their answers on the way back up (post-order). **In-order on a BST yields sorted order** – that one fact solves a whole family of BST problems.

TELL: a node's answer is built from its children's answers. Don't try to root it at a special node – let every node ask its children.

H = TREE HEIGHT – $O(\log n)$ balanced, $O(n)$ degenerate. That's the recursion stack.

$O(n)$ time / $O(h)$ space

17. Backtracking. [NEW] CUE: "all subsets", "all permutations", "all combinations", "every valid arrangement", N -queens, sudoku.

DOES: choose → recurse → *un*-choose. Build a partial solution, abandon it the moment it can't work (prune).

TELL: they want **every** solution enumerated, not a count or a best. Small n (≤ 20 ish) confirms it.

SAY THE EXPONENTIAL OUT LOUD – "this is $O(2^n)$ and that's inherent, there are 2^n subsets to emit." Naming it is the signal; apologizing for it is not.

$O(2^n)$ subsets / $O(n!)$ permutations time · $O(n)$ recursion depth (+ output size)

18. Fast / slow pointers. [NEW] CUE: linked list + "does it have a cycle", "find the middle", "find the start of the loop", "n-th from the end".

DOES: one pointer moves 1 step, the other 2. They meet iff there's a cycle; when fast hits the end, slow is at the middle.

TELL: linked list + a positional question, and you're told **$O(1)$ space** (which rules out the hash-set answer).

$O(n)$ time / $O(1)$ space ← the $O(1)$ is the entire point

The trap pairs

Two problems that look the same; a single constraint decides. This is the real test. Orange = missed on Aug 19.

#3 Prefix-sum array vs #4 Prefix-sum+hashmap — missed twice now

Both use running totals. The decider: **do you know the endpoints?** Endpoints *known* ("sum of range [i,j]", repeated queries, 2D) → plain array, $O(1)$ per lookup. Endpoints *unknown* ("which subarrays sum to k?") → hashmap to find them in $O(n)$ instead of $O(n^2)$. Same $O(n)$ storage; opposite direction (range→sum vs sum→range). **If you answer "prefix sum," immediately ask yourself "array or hashmap?" — the bare word is only half an answer.**

#2 Sliding window vs #4 Prefix-sum+hashmap

Both scream "contiguous subarray." The decider: **are negatives allowed?** Non-negative → window (running quantity changes monotonically with window size). Negatives present → window is *dead* (adding can lower the sum, breaking that monotonicity) → prefix-sum + hashmap.

The shuffle test — #2 sliding window vs everything else

Before saying "sliding window," ask: **if I shuffled the input, would the answer change?** If *yes*, order matters and a window is plausible. If *no*, the problem isn't about contiguity at all and a window cannot be the answer — you probably want a heap (#9), a hashmap (#5), or a sort. "*K-th largest*" survives any shuffle → not a window.

#8 Binary search on answer vs #10 Monotonic stack

Both involve "bigger/smaller" language. The decider: **what are you comparing?** Comparing array elements to *each other* ("next greater element") → monotonic stack. Testing candidate *answers* against a budget or deadline ("is speed v fast enough?") → binary search on the answer. In #8 the array elements are never compared to each other at all.

#1 Two-pointer vs #12/13 BFS/DFS

"Find a pair" on a flat **array** = two-pointer. BFS/DFS only when there's a **graph or grid** (nodes, edges, neighbors). No graph → no BFS.

#7 Binary search vs #8 Binary search on answer

Searching for a value *in sorted data* = #7, $O(\log n)$. Searching for the smallest/largest *parameter that makes something feasible* = #8, $O(n \log R)$ (the array may be unsorted; you bisect the answer range, not the array).

#9 Heap vs full sort vs "keep a sorted vector"

"k-th largest" — sorting works and is a fine first answer ($O(n \log n)$). You only need the top k , so a heap ($O(n \log k)$) or quickselect ($O(n)$) is the faster gear. **But maintaining a sorted vector by inserting as you go is $O(n)$ per insert — $O(n^2)$ total, worse than just sorting.** The right instinct ("keep a running collection of the best ones") needs the right container.

#6 Sort+sweep vs #5 Hashmap

Intervals/overlap = order on a number line → **sort**. A hashmap has no order, so it can't help with overlap. If your

complexity instinct says " $n \log n$ ", that's the sort talking — follow it.

#12 BFS vs #13 DFS — on a tree specifically

Both traverse everything. The decider: **do you need distance or levels?** "Minimum depth", "level order", "right side view", "nodes at distance k " → BFS with a queue. "Height", "diameter", "path sum", "is it balanced", anything combining children's answers → DFS/#16 recursion. *Height via DFS is natural; minimum depth via DFS is a trap — BFS stops at the first leaf.*

#17 Backtracking vs #14 DP

Same recursive smell. The decider: **do they want every solution, or just the count/best one?** "List all subsets that sum to k " → backtracking, exponential and unavoidable. "How many subsets sum to k " / "the largest such subset" → DP, polynomial. Emitting solutions forces exponential; counting them does not.

Recursion depth — any of #13, #14, #16, #17

If n could be $1e5$, a recursive DFS **blows the stack**. Say so unprompted and offer the explicit-stack iterative version. Free signal, and it's a real production instinct — the interviewer notices.

C++ execution notes

Recognition gets you the approach; these are what actually bite while typing it.

The five that cost points

- `m[key]` on a map **inserts** a default value — silently corrupts "have I seen it?" logic. Read with `.contains()`, `.find()`, or `.at()`.
- `priority_queue` is a **max-heap by default** — the opposite of what top-k-largest needs.
- `v.size()` is **unsigned**: `v.size() - 1` on an empty vector is a huge number.
- `(lo + hi) / 2` overflows; `lo + (hi - lo) / 2` doesn't. Sums of many large ints → `long long`.
- Pass big containers by `const&`. Say it out loud; it's free signal.

```
// min-heap (for k LARGEST) — the default is a max-heap
priority_queue<int, vector<int>, greater<int>> pq;

// #8 binary search on the answer
int lo = 1, hi = *max_element(a.begin(), a.end());
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;      // never (lo+hi)/2
    if (feasible(mid)) hi = mid;      // works → try smaller
    else lo = mid + 1;
}
return lo;                          // lo == hi == answer

// #2 sliding window (variable size)
int left = 0, best = 0;
for (int right = 0; right < n; ++right) {
    add(a[right]);
    while (!ok()) remove(a[left++]);
    best = max(best, right - left + 1);
}

// #10 monotonic stack — indices, not values
stack<int> st;
for (int i = 0; i < n; ++i) {
    while (!st.empty() && a[i] > a[st.top()]) { ans[st.top()] = i - st.top(); st.pop(); }
    st.push(i);
}

// #16 tree recursion — height and diameter in one pass
int best = 0;
int height(TreeNode* t) {
    if (!t) return 0;
    int l = height(t->left), r = height(t->right);
    best = max(best, l + r);          // diameter through this node
    return 1 + max(l, r);
}

// #17 backtracking — choose / recurse / un-choose
void go(int i, vector<int>& cur) {
    if (i == n) { out.push_back(cur); return; }
    go(i + 1, cur);                  // skip
    cur.push_back(a[i]); go(i + 1, cur); cur.pop_back(); // take, then UN-take
}
```

```
// #18 fast/slow - cycle detection, O(1) space
ListNode *slow = head, *fast = head;
while (fast && fast->next) {
    slow = slow->next; fast = fast->next->next;
    if (slow == fast) return true;
}
return false;
```

Recognition self-quiz (do it in the dark)

Read the cue, name the pattern + number + **time and space** before peeking. Answers at the bottom.

1. "Longest substring with at most 2 distinct characters." ($n \leq 1e5$)
2. "Count subarrays summing to k ; array has negatives." ($n \leq 1e5$)
3. "Minimum number of moves for a knight to reach a square." (grid)
4. "Largest rectangle in a histogram."
5. " k -th smallest element in an unsorted array." ($n \leq 1e5$)
6. "Given sorted array, find if two values sum to t ."
7. "Number of islands in a grid of land/water."
8. "Smallest ship capacity to deliver all packages within D days."
9. "Are these parentheses/brackets valid?"
10. "How many distinct values appear?" (values up to $1e9$)
11. "Merge all overlapping meeting times."
12. "Coins of given denominations — fewest to make amount N ."
13. "Find the one number that appears once; all others appear twice."
14. "For each element, the next element greater than it."
15. "Shortest contiguous subarray with sum $\geq S$; all positive." ($n \leq 1e5$)
16. "Answer Q queries, each the sum of a fixed array over range $[i, j]$." (immutable)
17. "Diameter of a binary tree — longest path between any two nodes."
18. "Print every subset of a set of 15 distinct integers."
19. "Detect whether a linked list has a cycle, using $O(1)$ extra space."
20. "Minimum depth of a binary tree." (*careful — not the same tool as #17 above*)
21. "How many subsets sum to exactly k ?" ($n \leq 200$, $k \leq 1000$) (*careful — not the same tool as #18 above*)
22. "Slowest machine speed that still finishes all jobs by the deadline." ($n \leq 1e4$, values $\leq 1e9$)

Answers: 1) #2 sliding window · $O(n)/O(1)$ · 2) #4 prefix-sum+hashmap (negatives!) · $O(n)/O(n)$ · 3) #12 BFS · $O(V+E)/O(V)$ · 4) #10 monotonic stack · $O(n)/O(n)$ · 5) #9 heap/quickselect (sort = OK first answer) · $O(n \log k)/O(k)$ · 6) #1 two-pointer · $O(n)/O(1)$ · 7) #13 DFS/union-find · $O(V+E)/O(V)$ · 8) #8 binary search on the answer · $O(n \log R)/O(1)$ · 9) #11 stack · $O(n)/O(n)$ · 10) #5 hashmap (values huge → not an array) · $O(n)/O(n)$ · 11) #6 sort + sweep · $O(n \log n)/O(1)$ · 12) #14 DP · $O(n \cdot \text{amount})/O(\text{amount})$ · 13) #15 bit trick (XOR all) · $O(n)/O(1)$ · 14) #10 monotonic stack · $O(n)/O(n)$ · 15) #2 sliding window (positive → monotonic) · $O(n)/O(1)$ · 16) #3 prefix-sum array (known endpoints, repeated queries) · $O(n)$ build, $O(1)/\text{query}$ · 17) #16 tree recursion, post-order · $O(n)/O(h)$ · 18) #17 backtracking · $O(2^n)/O(n)$ depth · 19) #18 fast/slow pointers · $O(n)/O(1)$ · 20) #12 **BFS**, not DFS — first leaf reached wins · $O(V+E)/O(V)$ · 21) #14 **DP**, not backtracking — they want a count, not the subsets · $O(n \cdot k)/O(k)$ · 22) #8 binary search on the answer · $O(n \log R)/O(1)$.

If a cue doesn't instantly fire **the name, the number, and both complexities**, that's the one to drill. Recognition over recall — let it wash over you. Sleep when it's boring.